Bloque 3. Integración, optimización y virtualización

En este bloque se estudia cómo integrar Apache HTTP Server con Apache Tomcat para construir una infraestructura web en la que cada servidor desempeña una función específica. También se analizan los principales parámetros que permiten adaptar el funcionamiento de Tomcat a las necesidades de una aplicación, así como los procedimientos para comprobar su disponibilidad, analizar su rendimiento y detectar posibles problemas.

Finalmente, se introduce la virtualización mediante máquinas virtuales y contenedores, prestando especial atención a Docker como herramienta para desplegar y distribuir aplicaciones web.

1. Integración de Apache HTTP Server con Tomcat

1.1. Arquitectura con servidor web y servidor de aplicaciones

Apache HTTP Server y Apache Tomcat son dos servidores que pueden trabajar conjuntamente para ofrecer una infraestructura web más flexible y segura. Aunque ambos pueden atender peticiones HTTP, sus funciones principales son diferentes.

- **Apache HTTP Server:** recibe las peticiones de los clientes, sirve contenidos estáticos, gestiona los dominios y puede encargarse de la terminación HTTPS.
- **Apache Tomcat:** ejecuta aplicaciones Java basadas en tecnologías como Servlets y JSP.

En una arquitectura integrada, Apache HTTP Server actúa como punto de entrada y reenvía determinadas peticiones a Tomcat. Esta técnica se conoce como **proxy inverso**.

Ejemplo de arquitectura



El usuario accede al servidor Apache. Este recibe la petición y, si corresponde a una aplicación Java, la reenvía a Tomcat. La respuesta vuelve a Apache y posteriormente al navegador.

1.2. ¿Qué es un proxy inverso?

Un proxy inverso es un servidor que recibe las peticiones de los clientes y las distribuye hacia uno o varios servidores internos. El cliente no necesita conocer la dirección real del servidor que ejecuta la aplicación.

En una instalación con Apache y Tomcat, el proxy inverso permite que las aplicaciones Java sean accesibles a través de Apache sin exponer directamente el puerto de Tomcat.

Entre sus ventajas destacan:

- Centralizar el acceso a diferentes aplicaciones web.
- Gestionar las conexiones HTTPS desde Apache.
- Ocultar los puertos internos de los servidores de aplicaciones.
- Distribuir peticiones entre varios servidores.
- Aplicar políticas de acceso y filtrado.

- Servir contenidos estáticos directamente desde Apache.

1.3. Módulos proxy de Apache

Apache HTTP Server dispone de diferentes módulos que permiten implementar un proxy inverso.

Módulo	Función
`mod_proxy`	Proporciona la funcionalidad básica de proxy.
`mod_proxy_http`	Permite reenviar peticiones mediante el protocolo HTTP.
`mod_ssl`	Permite configurar conexiones HTTPS.
`mod_headers`	Permite modificar y añadir cabeceras HTTP.

En Debian, los módulos se pueden habilitar mediante el comando `a2enmod`.

Habilitar los módulos necesarios

```
sudo a2enmod proxy
sudo a2enmod proxy_http
sudo systemctl restart apache2
```

El primer módulo activa la funcionalidad general de proxy y el segundo permite utilizar HTTP como protocolo de comunicación con Tomcat.

1.4. Configuración del proxy inverso

Apache permite definir reglas para indicar qué peticiones deben enviarse a Tomcat. Para ello se utilizan principalmente las directivas `ProxyPass` y `ProxyPassReverse`.

- **ProxyPass:** establece la correspondencia entre una ruta solicitada por el cliente y una dirección del servidor interno.
- **ProxyPassReverse:** modifica determinadas cabeceras de respuesta para que las redirecciones del servidor interno sean coherentes con la dirección pública.

Ejemplo: integrar Apache con Tomcat

Supongamos que Tomcat está funcionando en el mismo equipo que Apache, escuchando en el puerto 8080. Queremos que los usuarios puedan acceder a una aplicación desplegada en Tomcat mediante la ruta `/aplicacion`.

Podemos crear un archivo de configuración para el sitio web: `/etc/apache2/sites-available/aplicacion.conf`.

```
<VirtualHost *:80>

    ServerName aplicacion.local

    ProxyPreserveHost On

    ProxyPass /aplicacion http://127.0.0.1:8080/aplicacion
    ProxyPassReverse /aplicacion http://127.0.0.1:8080/aplicacion

    ErrorLog ${APACHE_LOG_DIR}/aplicacion_error.log
    CustomLog ${APACHE_LOG_DIR}/aplicacion_access.log combined

</VirtualHost>
```

La directiva `ProxyPreserveHost On` permite conservar la cabecera Host original del cliente al reenviar la petición.

Las dos directivas ProxyPass establecen que las peticiones dirigidas a `/aplicacion` se envíen al contexto correspondiente de Tomcat.

Para activar el sitio:

```
sudo a2ensite aplicacion.conf
sudo apache2ctl configtest
sudo systemctl reload apache2
```

La comprobación de la configuración debe mostrar `Syntax OK` antes de recargar el servicio.

1.5. Diferentes aplicaciones y servidores

Un mismo servidor Apache puede distribuir las peticiones entre diferentes aplicaciones desplegadas en Tomcat.

Ejemplo de distribución por rutas

```
ProxyPass /tienda http://127.0.0.1:8080/tienda
ProxyPassReverse /tienda http://127.0.0.1:8080/tienda

ProxyPass /gestion http://127.0.0.1:8080/gestion
ProxyPassReverse /gestion http://127.0.0.1:8080/gestion
```

En este caso, Apache reenvía las peticiones de las rutas `/tienda` y `/gestion` a las aplicaciones correspondientes de Tomcat.

Importante: si Apache y Tomcat están instalados en máquinas diferentes, la dirección `127.0.0.1` debe sustituirse por la dirección IP o el nombre del servidor donde se encuentra Tomcat. También es necesario permitir la comunicación entre ambos equipos en el puerto correspondiente.

1.6. Seguridad en la integración

Cuando Apache actúa como proxy inverso, conviene evitar que los usuarios accedan directamente a Tomcat si no es necesario.

Algunas medidas recomendables son:

- Permitir el acceso público únicamente a los puertos necesarios de Apache, normalmente 80 y 443.
- Limitar el acceso al puerto 8080 de Tomcat a las direcciones IP autorizadas.
- Utilizar HTTPS para proteger las comunicaciones entre el navegador y Apache.
- Evitar publicar interfaces de administración como Manager y Host Manager.
- Revisar las cabeceras HTTP y las reglas de acceso.

2. Ajustes y optimización del servidor Tomcat

2.1. Importancia de la configuración

La configuración de un servidor de aplicaciones influye directamente en su rendimiento, estabilidad y capacidad para atender peticiones.

Una configuración adecuada debe tener en cuenta los recursos disponibles en el sistema, las características de la aplicación y el número de usuarios que se espera que accedan al servicio.

Entre los parámetros más importantes se encuentran:

- Memoria asignada a la máquina virtual Java.
- Variables de entorno.
- Parámetros de ejecución de Tomcat.
- Configuración de conexiones y tiempos de espera.
- Gestión de errores y registros.

2.2. Memoria de la máquina virtual Java

Tomcat se ejecuta sobre la máquina virtual Java (JVM). Esta máquina virtual utiliza una zona de memoria denominada **heap**, destinada principalmente a almacenar los objetos creados por las aplicaciones Java.

La memoria disponible para la JVM puede configurarse mediante parámetros de ejecución.

Parámetro	Descripción
`-Xms`	Establece el tamaño inicial del heap.
`-Xmx`	Establece el tamaño máximo del heap.
`-Xss`	Establece el tamaño de la pila de cada hilo.
`-XX:+UseG1GC`	Activa el recolector de basura G1.

Ejemplo de configuración de memoria

```
-Xms512m
-Xmx2048m
-XX:+UseG1GC
```

En este ejemplo:

- La JVM comienza utilizando un heap de 512 MB.
- El heap puede crecer hasta un máximo de 2048 MB.
- Se utiliza el recolector de basura G1.

Los valores deben adaptarse a la memoria física disponible y a las necesidades reales de la aplicación. Asignar demasiada memoria a Tomcat puede perjudicar al resto de procesos del sistema.

2.3. Variables de entorno y parámetros de ejecución

Las variables de entorno permiten definir información que los programas utilizan durante su ejecución. En Java y Tomcat se emplean para indicar rutas de instalación, opciones de la JVM y otros parámetros.

Variable	Descripción
`JAVA_HOME`	Directorio donde está instalada la máquina virtual Java o el JDK.
`CATALINA_HOME`	Directorio de instalación de Tomcat.
`CATALINA_BASE`	Directorio que contiene la configuración y los datos de una instancia de Tomcat.
`JAVA_OPTS`	Opciones de la JVM utilizadas por los scripts de Tomcat que las admiten.

En instalaciones tradicionales de Tomcat, los scripts pueden utilizar variables como `JAVA_OPTS` y `CATALINA_OPTS`. Esta última suele reservarse para parámetros de la JVM aplicados al proceso servidor.

Ejemplo de parámetros de ejecución

```
export JAVA_HOME=/usr/lib/jvm/java-21-openjdk-amd64

export CATALINA_OPTS="-Xms512m -Xmx2048m"

$CATALINA_HOME/bin/startup.sh
```

El ejemplo muestra una forma de definir variables en una instalación manual de Tomcat. La ruta de Java debe adaptarse a la versión instalada.

2.4. Configuración en Debian

En Debian, cuando Tomcat se instala mediante los paquetes de la distribución, la estructura de directorios puede ser diferente de la instalación manual descargada desde la página oficial.

En una instalación habitual de Tomcat 10 mediante paquetes Debian, algunos directorios importantes son:

Ruta	Contenido
`/etc/tomcat10/`	Archivos de configuración.
`/var/lib/tomcat10/`	Datos y aplicaciones desplegadas.
`/var/log/tomcat10/`	Archivos de registro.
`/usr/share/tomcat10/`	Bibliotecas y archivos compartidos del paquete.

En estas instalaciones, los parámetros de ejecución de Java suelen gestionarse mediante la configuración del servicio y los mecanismos propios del paquete Debian. No se debe asumir que los scripts o rutas de una instalación manual existen en la instalación empaquetada.

Para consultar el servicio:

```
sudo systemctl status tomcat10
```

Para consultar su configuración de servicio:

```
systemctl cat tomcat10
```

También se pueden consultar los parámetros de la JVM del proceso en ejecución utilizando herramientas como `ps`:

```
ps aux | grep '[t]omcat'
```

> **Nota:** las rutas, los nombres de los servicios y los mecanismos para establecer variables pueden cambiar según la versión de Tomcat y la distribución Linux. Antes de modificar la configuración, conviene comprobar cómo se ha instalado el servidor.

2.5. Gestión de conexiones y tiempos de espera

Tomcat utiliza conectores para recibir las peticiones de los clientes. Estos conectores se configuran en el archivo `server.xml`.

Entre sus parámetros se encuentran:

- `port`: puerto en el que escucha el conector.
- `connectionTimeout`: tiempo máximo de espera para recibir la petición inicial.
- `maxThreads`: número máximo de hilos de procesamiento que pueden utilizarse para atender peticiones simultáneas en el conector.
- `maxConnections`: número máximo de conexiones que puede gestionar el conector.
- `acceptCount`: longitud máxima de la cola de peticiones pendientes cuando se alcanza el límite de conexiones.

Ejemplo de configuración de un conector HTTP

```
<Connector
    port="8080"
    protocol="HTTP/1.1"
    connectionTimeout="20000"
    maxThreads="150"
    maxConnections="8192"
    acceptCount="100"
/>
```

En este ejemplo se configura un conector HTTP en el puerto 8080. Se establece un límite de 150 hilos de procesamiento y una cola de hasta 100 peticiones pendientes cuando se alcanza el límite de conexiones.

Estos valores son orientativos. Su modificación debe realizarse después de analizar el comportamiento de la aplicación y los recursos disponibles.

2.6. Gestión de errores

Durante la ejecución de una aplicación pueden producirse errores de diferentes tipos:

- Errores de programación en el código Java.
- Problemas de conexión con bases de datos.
- Falta de memoria disponible.
- Errores en la configuración de Tomcat.
- Aplicaciones que no se despliegan correctamente.
- Problemas de permisos sobre archivos y directorios.

Es importante distinguir entre los errores que recibe el usuario y la información técnica que necesita el administrador para localizar su origen.

Una aplicación puede configurar páginas de error personalizadas mediante el descriptor `web.xml`.

Ejemplo de páginas de error personalizadas

```
<web-app>

    <error-page>
        <error-code>404</error-code>
        <location>/errores/404.jsp</location>
    </error-page>

    <error-page>
        <error-code>500</error-code>
        <location>/errores/500.jsp</location>
    </error-page>

</web-app>
```

En este caso, la aplicación utiliza páginas propias para los errores HTTP 404 (recurso no encontrado) y 500 (error interno del servidor).

Las páginas de error destinadas al usuario no deben mostrar rutas internas, trazas de ejecución, contraseñas ni información sensible.

2.7. Registros y diagnóstico

Los registros o logs permiten conocer qué sucede durante la ejecución de un servidor. Son fundamentales para detectar errores, estudiar el comportamiento de las aplicaciones y analizar posibles problemas de rendimiento.

En Tomcat pueden encontrarse diferentes tipos de registros:

Registro	Información habitual
`catalina.out`	Salida estándar y mensajes del proceso en determinadas instalaciones.
`catalina.YYYY-MM-DD.log`	Mensajes generales del servidor.
`localhost.YYYY-MM-DD.log`	Mensajes asociados al contexto local.
`localhost_access_log.YYYY-MM-DD.txt`	Peticiones HTTP recibidas, si está habilitado el registro de acceso.

Los nombres y la ubicación de los archivos dependen de la configuración y del mecanismo de instalación.

Consultar los registros en Debian

```
# Consultar los archivos de registro
ls -l /var/log/tomcat10/

# Consultar los últimos mensajes del servicio
sudo journalctl -u tomcat10 -n 50

# Seguir los mensajes en tiempo real
sudo journalctl -u tomcat10 -f
```

El parámetro `-n 50` muestra los últimos 50 mensajes. La opción `-f` permite observar los mensajes que se generan mientras el servicio está funcionando.

3. Pruebas funcionales y de rendimiento

3.1. Importancia de las pruebas

Una vez desplegada una aplicación, es necesario comprobar que funciona correctamente y que el servidor puede atender las peticiones de los usuarios.

Las pruebas permiten detectar errores antes de poner una aplicación en producción y comprobar que los cambios de configuración no afectan negativamente al servicio.

Se pueden distinguir dos grandes grupos:

- **Pruebas funcionales:** verifican que la aplicación realiza correctamente las operaciones previstas.
- **Pruebas de rendimiento:** analizan el comportamiento del sistema cuando recibe diferentes cantidades de peticiones.

3.2. Comprobación de disponibilidad

La disponibilidad indica si un servicio está operativo y puede atender las peticiones de los usuarios.

Una primera comprobación consiste en verificar que el servicio está iniciado.

```
sudo systemctl status tomcat10
```

También se puede comprobar si el servidor está escuchando en el puerto configurado:

```
sudo ss -ltnp | grep 8080
```

Para realizar una petición HTTP desde la terminal se puede utilizar `curl`.

Comprobar una respuesta HTTP

```
# Consultar únicamente las cabeceras HTTP
curl -I http://localhost:8080/

# Consultar el código de respuesta
curl -s -o /dev/null -w "%{http_code}\n" \
http://localhost:8080/
```

El código HTTP permite identificar el resultado de la petición. Algunos códigos habituales son:

Código	Significado
200	Petición correcta.
301 / 302	Redirección.
400	Petición incorrecta.
401	Se requiere autenticación.
403	Acceso prohibido.
404	Recurso no encontrado.
500	Error interno del servidor.
503	Servicio no disponible.

3.3. Pruebas funcionales

Las pruebas funcionales verifican que las diferentes partes de una aplicación responden de acuerdo con los requisitos establecidos.

En una aplicación web desplegada en Tomcat se pueden comprobar aspectos como:

- Acceso a la página principal.
- Navegación entre las diferentes secciones.
- Funcionamiento de formularios.
- Autenticación de usuarios.
- Control de acceso según los roles.
- Persistencia de los datos.
- Funcionamiento de las sesiones.
- Acceso a los recursos estáticos.
- Comportamiento ante rutas inexistentes.

Estas pruebas pueden realizarse manualmente desde un navegador o mediante herramientas automatizadas.

3.4. Tiempos de respuesta

El tiempo de respuesta es el intervalo que transcurre desde que un cliente realiza una petición hasta que recibe la respuesta del servidor.

Es uno de los indicadores más importantes para evaluar la experiencia de uso de una aplicación web.

El tiempo de respuesta puede verse afectado por:

- La velocidad de procesamiento del servidor.
- La cantidad de usuarios simultáneos.
- La complejidad de las operaciones realizadas.
- Las consultas a bases de datos.
- La latencia de la red.
- La cantidad de datos transmitidos.
- La configuración de Apache y Tomcat.

Medir el tiempo de respuesta con curl

```
curl -s -o /dev/null \
-w "Código: %{http_code}\nTiempo total: %{time_total}s\n" \
http://localhost:8080/
```

La variable `time_total` muestra el tiempo total empleado en completar la petición, expresado en segundos.

Esta medición permite realizar comprobaciones sencillas, aunque no sustituye a una prueba de rendimiento con múltiples peticiones.

3.5. Pruebas de carga básicas

Una prueba de carga consiste en someter una aplicación a un número determinado de peticiones para observar cómo responde.

Su objetivo es comprobar si el servidor puede mantener un funcionamiento adecuado cuando aumenta el número de usuarios o peticiones simultáneas.

Para realizar estas pruebas existen herramientas específicas:

Herramienta	Características
ApacheBench (ab)	Permite generar peticiones HTTP y medir resultados de forma sencilla.
JMeter	Permite diseñar pruebas de carga y simular diferentes usuarios y escenarios.
wrk	Herramienta de línea de comandos para generar carga HTTP.

Ejemplo con ApacheBench

ApacheBench puede instalarse en Debian mediante:

```
sudo apt install apache2-utils
```

Una vez instalado, se puede realizar una prueba sencilla:

```
ab -n 100 -c 10 http://localhost:8080/
```

Los parámetros indican:

- `-n 100` : realizar un total de 100 peticiones.
- `-c 10` : mantener hasta 10 peticiones concurrentes.

Entre los resultados se pueden analizar:

- Tiempo total empleado.
- Peticiones por segundo.
- Tiempo medio por petición.
- Porcentaje de peticiones con errores.
- Distribución de los tiempos de respuesta.

Precaución: las pruebas de carga deben realizarse únicamente sobre servidores propios o sistemas para los que se tenga autorización. Una prueba mal dimensionada puede consumir todos los recursos disponibles y dejar el servicio sin capacidad para atender a otros usuarios.

3.6. Interpretación de los resultados

Obtener un número elevado de peticiones por segundo no significa necesariamente que una aplicación tenga un buen rendimiento.

Es necesario analizar diferentes indicadores de forma conjunta:

Indicador	Qué permite evaluar
Peticiones por segundo	Capacidad de procesamiento de solicitudes.
Tiempo medio de respuesta	Tiempo habitual necesario para responder.
Percentiles de latencia	Comportamiento de los tiempos de respuesta en diferentes porcentajes de peticiones.
Errores HTTP	Solicitudes que no han finalizado correctamente.
Uso de CPU	Consumo de capacidad de procesamiento.
Uso de memoria	Consumo de memoria del proceso y del sistema.

Para localizar un problema de rendimiento conviene comparar los resultados de las pruebas con los registros del servidor y con las métricas del sistema operativo.

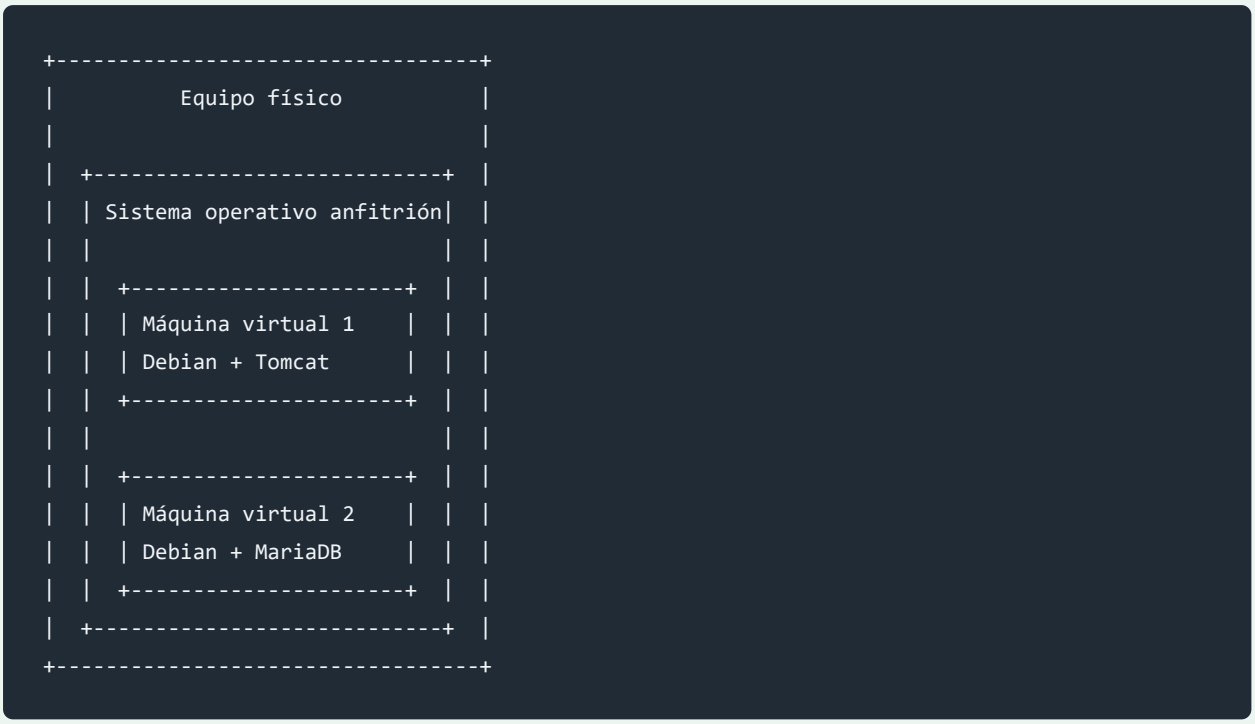
4. Introducción a la virtualización y los contenedores

4.1. Concepto de virtualización

La virtualización permite ejecutar entornos informáticos independientes sobre una misma máquina física. De esta forma, los recursos de hardware pueden compartirse entre diferentes sistemas.

Una máquina virtual simula un equipo completo, incluyendo procesador, memoria, almacenamiento y dispositivos de red. Sobre ella se instala un sistema operativo invitado que funciona de manera independiente del sistema anfitrión.

Ejemplo de virtualización



Herramientas como VirtualBox, VMware o KVM permiten crear y administrar máquinas virtuales.

4.2. Contenedores

Los contenedores son una tecnología de virtualización a nivel de sistema operativo que permite ejecutar aplicaciones en entornos aislados.

A diferencia de una máquina virtual tradicional, un contenedor no necesita incluir un sistema operativo completo. Comparte el kernel del sistema anfitrión y utiliza mecanismos de aislamiento para separar los procesos, redes y sistemas de archivos.

Los contenedores permiten empaquetar una aplicación junto con sus dependencias y configuración para facilitar su despliegue en diferentes entornos.

4.3. Diferencias entre máquinas virtuales y contenedores

Característica	Máquina virtual	Contenedor
Sistema operativo	Incluye un sistema operativo invitado.	Comparte el kernel del anfitrión.
Aislamiento	A nivel de máquina virtual.	A nivel de procesos y recursos.
Consumo de recursos	Generalmente mayor.	Generalmente menor.
Tiempo de inicio	Normalmente más elevado.	Normalmente más reducido.
Portabilidad	Permite trasladar máquinas virtuales completas.	Permite distribuir imágenes de aplicaciones.
Uso habitual	Ejecutar sistemas operativos completos.	Desplegar aplicaciones y servicios.

4.4. Docker

Docker es una plataforma que permite crear, distribuir y ejecutar aplicaciones mediante contenedores.

Sus elementos principales son:

- **Imagen:** plantilla que contiene los archivos y dependencias necesarios para ejecutar una aplicación.
- **Contenedor:** instancia en ejecución de una imagen.
- **Dockerfile:** archivo de instrucciones utilizado para construir una imagen.
- **Registro:** repositorio desde el que se pueden descargar y distribuir imágenes.
- **Volumen:** mecanismo para conservar datos fuera del ciclo de vida de un contenedor.
- **Red:** mecanismo que permite la comunicación entre contenedores y otros equipos.

4.5. Ejecutar Tomcat mediante Docker

Una de las ventajas de Docker es que permite desplegar Tomcat sin tener que instalar manualmente Java y el servidor de aplicaciones en el sistema anfitrión.

Se puede utilizar una imagen oficial de Tomcat disponible en Docker Hub.

Ejemplo: ejecutar un contenedor Tomcat

```
docker run -d \
    --name servidor-tomcat \
    -p 8080:8080 \
    tomcat:10.1
```

En este ejemplo:

- `docker run` crea e inicia un contenedor.
- `-d` ejecuta el contenedor en segundo plano.
- `--name` establece un nombre para el contenedor.
- `-p 8080:8080` publica el puerto 8080 del contenedor en el anfitrión.
- `tomcat:10.1` indica la imagen que se utilizará.

Después de iniciar el contenedor se puede consultar su estado:

```
docker ps
```

Y consultar los mensajes generados por Tomcat:

```
docker logs servidor-tomcat
```

La imagen seleccionada debe contener una aplicación de bienvenida accesible para que la comprobación desde el navegador muestre un resultado útil. La imagen oficial puede variar su contenido y estructura según la etiqueta utilizada.

4.6. Desplegar una aplicación WAR en Docker

Una aplicación Java empaquetada en formato WAR puede incorporarse a una imagen de Tomcat para que se despliegue al iniciar el contenedor.

Para ello se puede crear un archivo `Dockerfile` que utilice una imagen de Tomcat como base y copie el archivo WAR al directorio de aplicaciones.

Ejemplo de Dockerfile

Supongamos que el proyecto genera un archivo `aplicacion.war`.

```
FROM tomcat:10.1

COPY aplicacion.war /usr/local/tomcat/webapps/aplicacion.war
```

Para construir la imagen:

```
docker build -t mi-aplicacion-tomcat .
```

Para crear y ejecutar el contenedor:

```
docker run -d \
    --name mi-aplicacion \
    -p 8080:8080 \
    mi-aplicacion-tomcat
```

Si el despliegue se realiza correctamente, la aplicación estará disponible en:

```
http://localhost:8080/aplicacion/
```

Es importante que la versión de Tomcat sea compatible con la aplicación. Por ejemplo, Tomcat 10 utiliza las APIs `jakarta.servlet`, mientras que aplicaciones antiguas desarrolladas para Tomcat 9 pueden utilizar `javax.servlet`.

4.7. Docker Compose

Cuando una aplicación necesita varios servicios, como Tomcat y MariaDB, resulta útil definir toda la infraestructura en un único archivo.

Docker Compose permite describir y gestionar aplicaciones formadas por varios contenedores mediante un archivo YAML, habitualmente denominado `compose.yaml`.

Ejemplo: Tomcat y MariaDB con Docker Compose

El siguiente ejemplo define dos servicios: un servidor Tomcat y una base de datos MariaDB.

```
services:

  tomcat:
    image: tomcat:10.1
    container_name: servidor-tomcat
    ports:
      - "8080:8080"
    depends_on:
      - mariadb

  mariadb:
    image: mariadb:11
    container_name: servidor-mariadb
    environment:
      MARIADB_ROOT_PASSWORD: cambiar_esta_clave
      MARIADB_DATABASE: aplicacion
      MARIADB_USER: usuario_app
      MARIADB_PASSWORD: cambiar_esta_otra_clave
    volumes:
      - datos_mariadb:/var/lib/mysql

volumes:
  datos_mariadb:
```

En esta configuración:

- Se define un servicio llamado `tomcat` que publica el puerto 8080.
- Se define un servicio `mariadb` que crea una base de datos y un usuario inicial.
- Se utiliza un volumen para conservar los datos de MariaDB.
- `depends_on` establece una dependencia de arranque, pero no garantiza por sí sola que MariaDB esté preparada para aceptar conexiones.

Para iniciar los servicios:

```
docker compose up -d
```

Para consultar su estado:

```
docker compose ps
```

Para consultar los registros:

```
docker compose logs
```

Para detener y eliminar los contenedores y la red creada por Compose:

```
docker compose down
```

El volumen de datos no se elimina con el comando anterior. Para eliminar también los volúmenes definidos por Compose se utiliza `docker compose down -v`, por lo que debe emplearse con precaución.

Importante: las contraseñas del ejemplo son valores de demostración. En un entorno real no deben almacenarse directamente en archivos que se publiquen en repositorios. Es preferible utilizar variables de entorno protegidas, archivos de secretos o mecanismos específicos de gestión de credenciales.

4.8. Persistencia de datos y redes en contenedores

Los contenedores son entornos que pueden eliminarse y crearse de nuevo. Por este motivo, los datos que deban conservarse no deberían depender exclusivamente del sistema de archivos interno del contenedor.

Docker proporciona volúmenes que permiten almacenar información de forma independiente del ciclo de vida del contenedor.

También dispone de redes virtuales que permiten que los contenedores se comuniquen entre sí mediante sus nombres de servicio.

> **Comunicación entre Tomcat y MariaDB**
>
> En el ejemplo anterior, Tomcat y MariaDB pertenecen a la misma red creada automáticamente por Docker Compose.
>
> Por tanto, la aplicación Java puede utilizar el nombre del servicio `mariadb` como dirección del servidor de base de datos, en lugar de utilizar `localhost`.
>
> El puerto interno de MariaDB es el 3306. No es necesario publicar este puerto en el anfitrión para que Tomcat pueda conectarse a él dentro de la red de Compose.

5. Ejercicios prácticos

Los siguientes ejercicios permiten aplicar los conceptos estudiados en el bloque. Se recomienda documentar las configuraciones realizadas, las pruebas ejecutadas y los resultados obtenidos.

Ejercicio 1. Integración de Apache y Tomcat

1. Comprueba el funcionamiento de Apache HTTP Server y Tomcat.
2. Habilita los módulos necesarios para configurar un proxy inverso.
3. Configura Apache para acceder a una aplicación desplegada en Tomcat.
4. Comprueba el acceso desde un navegador utilizando Apache.
5. Analiza los registros generados por ambos servidores.

Ejercicio 2. Ajustes de Tomcat

1. Identifica los archivos y directorios de configuración de Tomcat.
2. Consulta los parámetros de ejecución de la JVM.
3. Modifica los parámetros de memoria del servidor.
4. Revisa los parámetros del conector HTTP.
5. Reinicia el servicio y comprueba que continúa funcionando correctamente.
6. Analiza los registros para identificar posibles errores.

Ejercicio 3. Pruebas de funcionamiento y rendimiento

1. Comprueba la disponibilidad del servidor mediante herramientas de terminal.
2. Realiza diferentes peticiones HTTP a una aplicación desplegada.
3. Mide los tiempos de respuesta.

4. Ejecuta una prueba de carga básica con ApacheBench.
5. Analiza las peticiones por segundo, los tiempos de respuesta y los errores.
6. Compara los resultados de varias ejecuciones.

Ejercicio 4. Despliegue mediante contenedores

1. Comprueba que Docker está instalado y funcionando.
2. Descarga una imagen de Tomcat.
3. Crea un contenedor y publica el puerto HTTP.
4. Despliega una aplicación WAR en una imagen personalizada.
5. Comprueba el funcionamiento de la aplicación desde el navegador.
6. Consulta los registros y el estado del contenedor.

Ejercicio 5. Infraestructura con Docker Compose

1. Define una infraestructura con Tomcat y MariaDB mediante Docker Compose.
2. Configura la red y la persistencia de los datos.
3. Comprueba que los servicios se inician correctamente.
4. Verifica la comunicación entre la aplicación y la base de datos.
5. Detén y vuelve a iniciar los servicios.
6. Comprueba que los datos almacenados en MariaDB se conservan.

6. Resumen del bloque

- Apache HTTP Server y Tomcat pueden integrarse mediante un proxy inverso para separar la recepción de peticiones de la ejecución de aplicaciones Java.
- Los módulos `mod_proxy` y `mod_proxy_http` permiten configurar el reenvío de peticiones HTTP.
- Los parámetros de la JVM permiten ajustar la memoria y otros aspectos de la ejecución de Tomcat.
- La configuración de conectores permite adaptar el servidor al número de conexiones y peticiones simultáneas.
- Los registros permiten diagnosticar errores y analizar el funcionamiento del servidor.
- Las pruebas funcionales comprueban que la aplicación realiza correctamente sus operaciones.
- Las pruebas de carga permiten analizar el rendimiento bajo diferentes niveles de actividad.
- Las máquinas virtuales proporcionan entornos completos y aislados, mientras que los contenedores permiten ejecutar aplicaciones compartiendo el kernel del sistema anfitrión.
- Docker facilita la creación, distribución y ejecución de aplicaciones mediante imágenes y contenedores.

- Docker Compose permite definir y administrar infraestructuras formadas por varios servicios.